



2026-Project-2: Domotics

Daniel Poli, Francesco Vaona and Tommaso Santus

Issue: 1

Date: June 24, 2026

University of Trento
Department of Information Engineering and Computer Science
Via Sommarive 9, 38123
Povo (TN), Italy

1 Introduction

This project implements a Domotic System (Smart home Automation System) multi-process and compliant at POSIX in C. The system consists in a central Controller and various independent Device processes (such as Bulbs, Windows, Fridges, Hubs and Timers). These processes use asynchronously communication using Named Pipes (FIFOs) and a simple pipe-delimited (—) text protocol. To avoid an high usage of CPU, the system uses the `select()` system call for I/O multiplexing together with a keepalive mechanism. The project include also an interactive Command-Line Interface and a manual override tool.

2 Process Lifecycle and Architectural Dichotomy

The foundational design choice of this project is the strict separation between the OS-level System Hierarchy and the application-level Logical Hierarchy. At the OS level, the Controller acts like the root process (PPID) for all the devices. The new devices gets a dynamical instance at runtime, using the standard system call POSIX `fork()` ed `exec1()` system calls. Thanks to binary polymorphism, the same compiled executable is reused to spawn different devices by passing specific command-line arguments (e.g., `-device-bulb`). The controller takes full responsibility for handling `SIGCHLD` signals and the collection of terminated processes via `waitpid()` , preventing the resource leaks.

Instead, the Logical hierarchy (e.g., a Hub managing several Bulbs) is purely virtual. It is maintained internally by the Controller though a routing table and `logical_parent_id` bindings. This division produces significant architectural benefits. Primarily, it ensures high fault tolerance: if a logical parent like a Hub crashes, its child devices are not lost or orphaned at the OS level; the Controller can adopt them without interruptions and updates the routing table without requiring process restarts. Furthermore, it allows for dynamic reconfiguration of the smart home layout without altering the underlying process tree, ensuring that the IPC names pipes remain stable throughout the system's executions

3 Mitigation of Orphan Processes and Race Conditions

A critical challenge in multi-process architectures is ensuring safe memory cleanup during unexpected terminations. To prevent devices from lingering as orphan processes in the background if the Controller crashes, the system uses the Linux-specific `prctl` system call. After that a `fork()` call gets completed successfully, the child process invokes `prctl(PR_SET_PDEATHSIG, SIGTERM)` . This teaches the kernel to automatically deliver a termination signal to the child if the parent thread group exists.

However, this approach introduces a race condition: if the Controller terminates exactly during the context switch between the `fork()` and the `prctl()` execution, the signal directive might not me registered. To neutralize this vulnerability, a retroactive check was implemented. The child explicitly verifies its parent PID (`if (getppid() == 1) { _exit(1); }`), forcing an immediate exit if it discovers it has already been adopted by the init process before the safety mechanism could be applied.

4 Inter-Process Communication (IPC) and Protocol Design

The system's inter-process communication relies exclusively on POSIX Named Pipes (FI-FOs), implementing a solid request-reply architectural pattern. Each spawned device

creates and maintains a persistent, static listening FIFO for incoming commands. To handle asynchronous requests that require a synchronous-like response (like polling a device's status) without blocking the main Controller execution, the system dynamically allocates temporary FIFOs paired with a unique request ID.

A notable technical challenge with non-blocking read operations on FIFOs is the continuous EOF (End-Of-File) signaling triggered when there are no writing process currently connected. This behavior would cause the `select()` system call to return immediately in a tight loop, leading to 100% CPU usage. Keepalive writer: To mitigate this issue, we implemented a “keepalive writer” pattern: the reading process silently opens its own FIFO in `(O_WRONLY | O_NONBLOCK)` mode (`fd_keep = open(path, O_WRONLY | O_NONBLOCK);`). This ensures that at least one write file descriptor remains open at all times, keeping the pipe active and the CPU idle.

Data serialization over these pipes is governed by a custom text-based protocol (`domo_message`), structured as a pipe-delimited string (e.g., `sender_i | command | target_id | ... | payload \n`). This ensures simple parsing, atomic writes, and cross-process compatibility.

5 Event Multiplexing and Non-Blocking I/O

To guarantee high responsiveness and avoid the complexities of multi-threading, the Controller implements a single-threaded, event-driven architecture centered around the `select()` system call. The event loop concurrently monitors three distinct I/O boundaries:

- the standard input (`STDIN_FILENO`) to process interactive user command via CLI;
- the Controller's main FIFO for unsolicited device status updates error reporting;
- a dynamically managed pool of file descriptors representing temporary FIFOs awaiting pending replies;

To prevent the Controller from blocking indefinitely during quiet periods, the `select()` call is limited by a `200ms` timeout constraint. This design choice allows the system to periodically break out of the waiting state to perform critical asynchronous maintenance tasks. These include expiring timed-out pending requests, cleaning up temporary files, and collection terminated child processes, ensuring maximum CPU efficiency and real-time operational stability.

6 Challenges encountered and solutions

The project required facing problems of concurrent IPC-based systems: non-deterministic latency, pending requests, graceful shutdown and logical hierarchy consistency. The use of `simulate_random_delay()` introduced variable response times across devices, so every architectural choice had to remain robust both in normal execution and when events happened simultaneously.

To keep the system understandable and testable, responsibilities were split across layers. The controller manages the shell and overall coordination, devices execute their local behavior and the routing layer preserves the logical structure independently from the real process tree, which is flat at OS level as required by specification.

6.1 Graceful shutdown

One of the first issues appeared around the `exit` command were with random delays enabled: some devices could still be preparing a response when the Controller started shutting down, so terminating everything immediately risked cutting off FIFOs too early, losing valid messages, and leaving the system in an inconsistent state.

The solution was to implement a gradual shutdown inside `event_loop_run()`, using `select()` with short timeouts even in the final phase. The main loop continuously monitors keyboard input, the Controller FIFO, and all pending reply FIFOs. When `exit` is requested, the Controller performs a small flush cycle to give active pipes time to drain before resources are released.

This approach reduced the IPC congestion and made the shutdown behavior much more stable under automatic testing. In addition, periodic `SIGCHLD` handling prevents zombie accumulation, because terminated children are explicitly reaped by the controller during loop execution.

6.2 Coherence management and the re-linking algorithm

The problems that had to be solved within the scope of the project include latency that cannot be predicted, requests that wait for an answer, the process of closing the system down safely, and the necessity to maintain the consistency of the logical hierarchy. As the usage of `simulate_random_delay()` results in different reaction times on different devices, it is crucial for all components of the architecture to work properly in both usual operation mode and when several events take place simultaneously.

The solution lay in making the link operation into a multiple step transaction. The Controller performs the operation as follows in `controller_link_device()` - update the routing table, notify the child about the parent change, notify the parent about the child addition, and notify the old parent about the removal of the child via `controller_notify_parent_child_removed()`. This makes sure that all parties involved update themselves about the logical network topology. This solution is also in line with the structural constraints introduced in the function `routing_link_devices()` that disallow self-connections, invalid parent connections, non-control parent connection, and loops in the logical hierarchy. Thus, the validation process is divided into two stages where the controller validates the user command, and the routing stage validates the logical consistency.

6.3 Safe Timer implementation

The Timer is a specialized control component that was developed with one child and only two defined schedule limits – `begin_time` and `end_time`. The function `validate_time_format()` will validate format of time strings by checking their length, : symbol usage, and the values of hours and minutes, while `compare_times()` will make sure that `begin_time` comes before `end_time`.

With both of these levels of security, the problem of inconsistent configurations being allowed into the internal state does not arise, but instead is prevented at the onset. Furthermore, `timer_update()` is invoked prior to message handling and again in the device loop; as such, the Timer object may choose to forward its internal state depending on the time.

The `manual_override` feature is an important one. When the child signals that there has been a manual override, the Timer ceases from operating in an automated manner and

does not impose any further automatic behavior on the branch state. Normal operation resumes only when the `SWITCH` command is issued again.

6.4 Bash testing infrastructure

The validation process for the system was performed using a set of Bash scripts that work with the Controller in the manner a human would. The scripts do not simply verify the last exit code but create commands, store the controller output in a file, and validate its behavior using `grep` and regular expressions.

As for the `test_linking.sh` script, command are piped but are separated by `sleep 2` from one to another. This is very important as each of the devices has random latency until the response. If there were no time difference between the commands, the system would be tested at the time when the commands are still being executed.

An additional delay is included in the script to allow enough time for the Controller to finish processing any remaining responses and empty FIFOs before closing down. This test takes into account both positive and negative scenarios, such as correct device association, invalid choice of parent, and device relink that preserves hierarchy observability.

6.5 Manual interaction and external override

An explicit manual interaction interface was created to provide simulation of the physical operation done on a device from another terminal. User side entry point for it is `manual_interaction.sh` bash wrapper script that takes a device ID and a command, while `manual_client.c` takes care of the actual IPC communication. Client checks its arguments, reads registry data to find the targeted device and makes a request right to the device FIFO.

Such approach allows keeping the manual way of interaction separate from the ordinary routing flow and meeting the condition of being out-of-band communication. An outgoing message is marked as an external one so that the targeted device could tell an override command from a Controller's one. It is very useful when doing tests of overrides and local state changes in logically controlled devices.

6.6 Process termination and crash handling

The occurrence of an unexpected termination, like `SIGKILL`, cannot be caught by the target process, and thus the Controller needs to realize the failure and modify its internal state. In this case, the device will get unlinked from the routing tree, requests depending on the crashed process will be marked as failures/expirations, and the related FIFOs/registry entries will be deleted. This ensures that the crashed device does not block the event loop or the logical hierarchy remains inconsistent.

Thus, failure handling can be considered as being part of the consistency model of the linking/deletion mechanism, where each failure corresponds to some controlled state transition.

7 Conclusions

These adopted design decisions helped create a system which proved itself to be much more resistant to the mentioned concurrency issues, which include user input and IPC data processing concurrently, gradual shutdown, termination reaping, logical consistency

of the data flow, and preventive validation of certain operations. These synchronized Bash tests helped maintain stability even in case of artificial delays.

As such, the overall project achieves all the objectives that have been set out in the specification, which include avoiding zombie processes in normal runs, avoiding any form of deadlock in the major processes, and exhibiting behavior according to the POSIX model. The strict separation of the process hierarchy and the logical hierarchy enabled the implementation of these operations in difficult conditions.